

STREAMFLOW

DOCUMENTAÇÃO TÉCNICA DO PROJETO

Banco de Dados II - Procedures, Functions, Triggers, Segurança e ORM

Aluno	João Victor Xavier da Silva
Turma	3 DS Ams
Professor	Salomão
Data	14/09/2026
Status	Versão final para entrega

Nota de ambiente

O script declara MySQL 8.0 como SGBD-alvo. Durante os testes integrados da camada ORM, o servidor real foi identificado como MariaDB. O relatório registra essa diferença explicitamente e trata a implementação como compatível com o dialeto MySQL/MariaDB utilizado no laboratório.

Controle do documento

Campo	Descrição
Projeto	StreamFlow - plataforma de streaming com controle de assinantes, perfis, conteúdo, reprodução, faturamento e auditoria
Disciplina	Banco de Dados II
Versão	1.0
SGBD-alvo	MySQL 8.0, conforme cabeçalho do script SQL
Ambiente validado	MariaDB compatível com o conjunto de recursos utilizados
ORM	Drizzle ORM + Drizzle Kit
Linguagem da aplicação	TypeScript / Node.js
Driver	mysql2
Status	Implementação validada; evidências visuais reservadas nos espaços indicados.

Resumo executivo

O StreamFlow foi evoluído de um modelo relacional tradicional para uma solução de banco de dados com regras de negócio centralizadas no SGBD, tratamento explícito de exceções, auditoria, segurança por privilégios e integração com uma aplicação por ORM. A solução mantém o banco como fonte de verdade para regras críticas, enquanto a camada TypeScript utiliza Drizzle ORM para mapeamento e consultas estruturadas.

A implementação final contém 12 tabelas de domínio, Stored Functions, Stored Procedures transacionais, cursor, handlers de exceção, triggers BEFORE e AFTER, índices de desempenho, uma view orientada à LGPD, dois usuários com perfis de acesso distintos e uma camada ORM gerada por introspecção do banco existente.

Resultado geral

Os testes SQL e os testes de integração com a aplicação validaram as Functions, as três Procedures, as regras de integridade, a imutabilidade dos logs, a auditoria, a atualização de timestamp, a normalização de nomes, o limite de perfis e o faturamento mensal. A camada ORM também foi validada com consultas e chamadas às rotinas armazenadas.

Sumário

- 1. Introdução e escopo
- 2. Arquitetura da solução
- 3. Modelo de dados
- 4. Stored Functions
- 5. Stored Procedures
- 6. Triggers e regras automatizadas
- 7. Tratamento de exceções e transações
- 8. Índices e desempenho
- 9. View LGPD
- 10. Segurança e privilégios
- 11. Dados de teste e consultas analíticas
- 12. Plano de validação e resultados
- 13. Mapeamento ORM com Drizzle
- 14. Integração das rotinas SQL pela aplicação
- 15. Compatibilidade MariaDB/MySQL e decisões técnicas
- 16. Matriz de rastreabilidade do enunciado

- 17. Guia de execução
- 18. Conclusão
- Apêndice A. Estrutura de entrega

1. Introdução e escopo

1.1 Contexto do projeto

O StreamFlow representa uma plataforma de streaming que administra assinantes, perfis, preferências, produtoras, conteúdos, filmes, séries, episódios, históricos de consumo e logs de reprodução. A evolução deste bimestre acrescenta lógica procedural e mecanismos de integridade diretamente no banco de dados, permitindo que regras críticas sejam protegidas independentemente da aplicação que consome os dados.

1.2 Objetivos técnicos

- Encapsular regras de cobrança, registro de reprodução e faturamento em Stored Procedures.
- Reutilizar cálculos por meio de Stored Functions, incluindo idade e minutos consumidos por produtora.
- Impedir violações de integridade por meio de CHECK constraints e triggers.
- Garantir atomicidade com transações, COMMIT, ROLLBACK e handlers de exceção.
- Registrar auditoria com valores OLD e NEW após alterações relevantes.
- Aplicar segurança por princípio do menor privilégio.
- Integrar o banco existente a uma aplicação TypeScript utilizando Drizzle ORM sem recriar a lógica de negócio no código da aplicação.

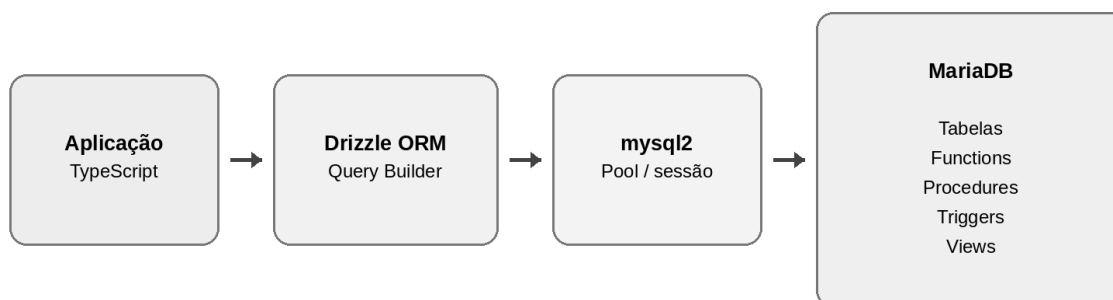
1.3 Escopo desta documentação

Este documento explica não apenas o que foi implementado, mas também por que cada decisão foi tomada, como os objetos se relacionam, quais riscos são mitigados, como os testes comprovam o comportamento esperado e como a camada ORM preserva o encapsulamento das regras mantidas no SGBD.

2. Arquitetura da solução

A arquitetura foi organizada em duas camadas principais: o banco relacional, responsável por persistência e regras críticas; e a aplicação TypeScript, responsável por consultas, integração e orquestração. O Drizzle ORM atua como camada de mapeamento e construção de consultas, enquanto o driver mysql2 gerencia a conexão e sessões necessárias para chamadas com parâmetros OUT.

Arquitetura lógica do StreamFlow



Princípio central: a aplicação acessa os dados pelo ORM, enquanto as regras críticas permanecem encapsuladas no SGBD.

Stored Procedures e Functions são invocadas pela camada de dados; transações, SIGNAL, handlers e cursor continuam no banco.

Figura 1 - Arquitetura lógica da integração entre aplicação, ORM e SGBD.

2.1 Princípio de encapsulamento

As regras de negócio críticas não foram duplicadas na aplicação. A cobrança mensal continua sendo validada e executada por `realizar_cobranca_mensal`; a inclusão de logs continua sendo controlada por `registrar_reproducao`; e o faturamento continua sendo calculado por `gerar_faturamento_mensal`, que reutiliza a Function `minutos_assistidos_por_produtores`. A aplicação apenas fornece parâmetros, invoca as rotinas e consulta os resultados.

2.2 Estrutura lógica da camada ORM

```

orm/
├── drizzle/
│   ├── <baseline gerado pela introspecção>/
│   ├── migration.sql
│   └── snapshot.json
├── src/
│   ├── db/
│   │   ├── index.ts
│   │   ├── schema.ts
│   │   └── relations.ts
│   ├── consultas.ts
│   └── procedimentos.ts
├── .env.example
├── .gitignore
├── drizzle.config.ts
├── package.json
├── package-lock.json
└── tsconfig.json
  
```

Estrutura recomendada da entrega ORM.

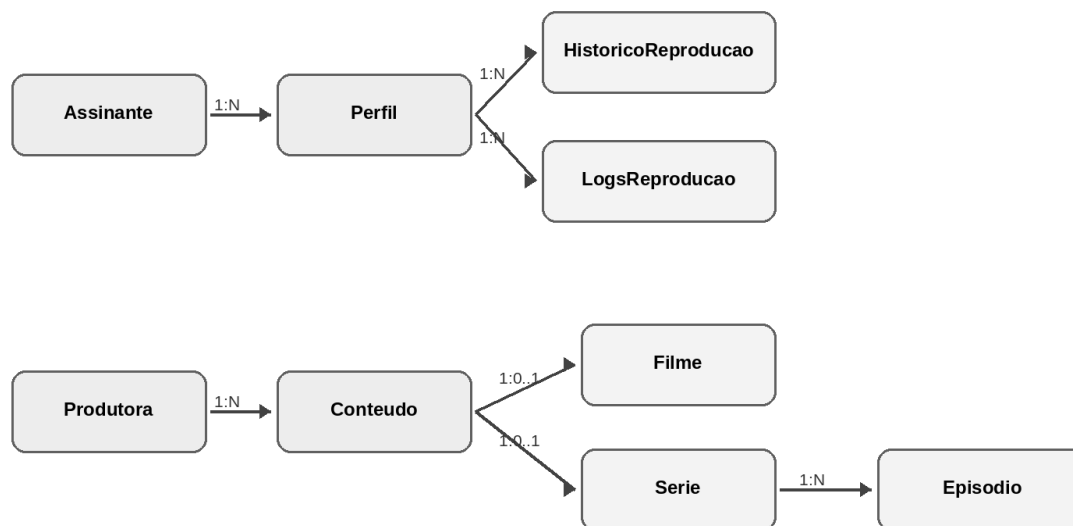
Prática de segurança

O arquivo `.env` real contém credenciais locais e não deve ser enviado ou versionado. A entrega deve conter apenas `.env.example` com valores fictícios ou placeholders.

3. Modelo de dados

O modelo mantém separação entre cadastro, catálogo, consumo, faturamento e auditoria. A divisão reduz acoplamento e permite que cada conjunto de dados tenha regras próprias de integridade.

Visão conceitual dos relacionamentos principais



Especialização lógica: Conteudo identifica o tipo; Filme e Serie refinam os atributos específicos.

Figura 2 - Visão conceitual dos relacionamentos principais.

Tabela	Responsabilidade	Observação
assinantes	Cadastro do titular, identificação, UF, data de nascimento e saldo.	Perfis; cobrança; LGPD.
perfis	Subidentidades de uso vinculadas a um assinante.	Máximo de 5 por assinante; auditoria.
preferencias_perfis	Categoria favorita de cada perfil.	Relacionamento 1:N com perfis.
produtoras	Empresas proprietárias dos conteúdos.	Conteúdos e faturamento.
conteudos	Entidade-base de catálogo.	Especializa em filme ou série.
filmes	Dados específicos de filmes.	Duração e reproduções.
series	Dados específicos de séries.	Relaciona-se a episódios.
episodios	Unidades de uma série.	Temporada, número, duração e reproduções.
historicos_reproducao	Estado de progresso para “continuar assistindo”.	Pode ser atualizado; registra progresso.
logs_reproducao	Eventos imutáveis de reprodução.	Fonte de consumo, auditoria e faturamento.
faturamento_produtoras	Consolidação mensal por produtora.	Competência + minutos consumidos.
auditoria_log	Registro de alterações auditadas.	Usuário, OLD/NEW e timestamp.

3.1 Especialização de conteúdo

A tabela conteudos concentra os atributos comuns do catálogo, como título, gênero, classificação indicativa, produtora e tipo. O campo tipo_conteudo identifica se o registro é FILME

ou SERIE. A tabela filmes possui uma relação 1:1 lógica com conteúdos por `conteudo_id` UNIQUE; a tabela series segue a mesma estratégia e se relaciona em 1:N com episódios.

Decisão de modelagem

A especialização evita duplicar atributos comuns entre filmes e séries. O ORM reflete essa estrutura por relações entre Conteúdo, Filme, Serie e Episódio, preservando a modelagem original.

3.2 Histórico x log de reprodução

`historicos_reproducao` e `logs_reproducao` representam conceitos distintos. O histórico registra o estado atual do consumo e pode mudar conforme o usuário avança; o log registra eventos de reprodução e é tratado como imutável. Essa distinção permite oferecer “continuar assistindo” sem perder uma trilha confiável de consumo utilizada em análises e faturamento.

3.3 Integridade declarativa

O schema utiliza chaves primárias, chaves estrangeiras, UNIQUE e CHECK constraints. Entre as restrições mais relevantes estão `saldo >= 0`, duração positiva, minutos não negativos, unicidade de episódio por série/temporada/número e coerência entre `tipo_midia`, `filme_id` e `episodio_id`.

4. Stored Functions

4.1 Function calcular_idade

A Function `calcular_idade` recebe uma data de nascimento e devolve a idade completa em anos. Ela utiliza `TIMESTAMPDIFF(YEAR, data, CURDATE())`, evitando que cada consulta repita a mesma lógica.

A declaração `NOT DETERMINISTIC` é apropriada porque o resultado pode mudar com o passar do tempo mesmo quando a data de nascimento de entrada permanece igual. A cláusula `NO SQL` informa que a Function não lê nem altera tabelas; ela depende apenas do parâmetro e da data corrente do servidor.

```
CREATE FUNCTION calcular_idade(
    p_data_nascimento DATE
)
RETURNS INT
NOT DETERMINISTIC
NO SQL
BEGIN
    DECLARE v_idade INT;

    IF p_data_nascimento IS NULL THEN
        RETURN NULL;
    END IF;

    SET v_idade = TIMESTAMPDIFF(
        YEAR,
        p_data_nascimento,
        CURDATE()
    );

    RETURN v_idade;
END$$
```

Código 1 - Stored Function calcular_idade.

Elemento	Descrição
Entrada	p_data_nascimento DATE
Retorno	INT - idade completa em anos
Classificação	NOT DETERMINISTIC / NO SQL
Uso principal	View LGPD e consultas de engajamento por faixa etária

4.2 Function minutos_assistidos_por_produtores

Essa Function consolida o total de minutos consumidos de uma produtora em uma competência mensal. Como o modelo separa filmes e episódios, a lógica executa duas agregações independentes e soma os resultados.

O início do mês é normalizado para o primeiro dia da competência e o limite superior é calculado adicionando um mês. O uso do intervalo [início, fim) evita problemas com horários e funciona corretamente para qualquer quantidade de dias no mês.

```
CREATE FUNCTION minutos_assistidos_por_produtores(
    p_produtores_id BIGINT,
    p_competencia DATE
)
RETURNS BIGINT
NOT DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_inicio DATE;
    DECLARE v_fim DATE;
    DECLARE v_minutos_filmes BIGINT DEFAULT 0;
    DECLARE v_minutos_episodios BIGINT DEFAULT 0;

    SET v_inicio = STR_TO_DATE(
```



```

        DATE_FORMAT(p_competencia, '%Y-%m-01'),
        '%Y-%m-%d'
    );

    SET v_fim = DATE_ADD(
        v_inicio,
        INTERVAL 1 MONTH
    );

    SELECT COALESCE(SUM(l.tempo_assistido_minutos), 0)
    INTO v_minutos_filmes
    FROM logs_reproducao l
    INNER JOIN filmes f
        ON f.id = l.filme_id
    INNER JOIN conteudos c
        ON c.id = f.conteudo_id
    WHERE c.produtora_id = p_produtora_id
        AND l.tipo_midia = 'FILME'
        AND l.data_hora_acesso >= v_inicio
        AND l.data_hora_acesso < v_fim;

    SELECT COALESCE(SUM(l.tempo_assistido_minutos), 0)
    INTO v_minutos_episodios
    FROM logs_reproducao l
    INNER JOIN episodios e
        ON e.id = l.episodio_id
    INNER JOIN series s
        ON s.id = e.serie_id
    INNER JOIN conteudos c
        ON c.id = s.conteudo_id
    WHERE c.produtora_id = p_produtora_id
        AND l.tipo_midia = 'EPISODIO'
        AND l.data_hora_acesso >= v_inicio
        AND l.data_hora_acesso < v_fim;

    RETURN v_minutos_filmes + v_minutos_episodios;
END$$

```

Código 2 - Stored Function minutos_assistidos_por_produtora.

Decisão	Justificativa
READS SQL DATA	A Function consulta logs, filmes, episódios, séries e conteúdos.
NOT DETERMINISTIC	O resultado depende do estado atual das tabelas e pode mudar quando novos logs são inseridos.
COALESCE(SUM(...), 0)	Garante retorno zero quando não houver consumo no período.
Separação filme/episódio	Respeita a especialização do catálogo e evita joins ambíguos.

5. Stored Procedures

5.1 Procedure realizar_cobranca_mensal

A Procedure de cobrança implementa a RN01 em fluxo transacional. Ela recebe o assinante e o valor da mensalidade, valida os parâmetros, bloqueia a linha do assinante com FOR UPDATE, verifica saldo suficiente, realiza o débito e retorna o novo saldo em um parâmetro OUT.

Parâmetro	Modo	Tipo	Função
p_id_assinante	IN	BIGINT	Identifica o assinante.
p_valor_mensalidade	IN	DECIMAL(12,2)	Valor a debitar.
p_novo_saldo	OUT	DECIMAL(12,2)	Saldo final após a operação.

```
CREATE PROCEDURE realizar_cobranca_mensal(
    IN p_id_assinante BIGINT,
    IN p_valor_mensalidade DECIMAL(12,2),
    OUT p_novo_saldo DECIMAL(12,2)
)
BEGIN
    DECLARE v_saldo_atual DECIMAL(12,2);
    DECLARE v_existe INT DEFAULT 0;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;

    IF p_valor_mensalidade IS NULL OR p_valor_mensalidade <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'O valor da mensalidade deve ser maior que zero.';
    END IF;

    SELECT COUNT(*)
    INTO v_existe
    FROM assinantes
    WHERE id = p_id_assinante;

    IF v_existe = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Assinante nao encontrado.';
    END IF;

    SELECT saldo
    INTO v_saldo_atual
    FROM assinantes
    WHERE id = p_id_assinante
    FOR UPDATE;

    IF v_saldo_atual < p_valor_mensalidade THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Saldo insuficiente para realizar a cobranca.';
    END IF;

    UPDATE assinantes
    SET saldo = saldo - p_valor_mensalidade
    WHERE id = p_id_assinante;

    SELECT saldo
    INTO p_novo_saldo
    FROM assinantes
    WHERE id = p_id_assinante;

    COMMIT;
END$$
```

Código 3 - Procedure realizar_cobranca_mensal.

Fluxo de execução

1. Inicia transação.
2. Valida se o valor é maior que zero.
3. Valida a existência do assinante.
4. Lê e bloqueia o saldo com SELECT ... FOR UPDATE.
5. Emite SIGNAL se o saldo for insuficiente.
6. Atualiza o saldo.
7. Retorna o novo saldo pelo parâmetro OUT.
8. Executa COMMIT.
9. Qualquer erro SQL aciona ROLLBACK e RESIGNAL.

5.2 Procedure registrar_reproducao

A Procedure registrar_reproducao cria um novo evento em logs_reproducao. Ela valida perfil, tipo de mídia, dispositivo e existência do filme ou episódio. O registro é criado com zero minutos e o identificador gerado é devolvido por LAST_INSERT_ID().

Parâmetro	Modo	Responsabilidade
p_perfil_id	IN	Perfil responsável pela reprodução.
p_tipo_midia	IN	FILME ou EPISODIO.
p_midia_id	IN	ID do filme ou episódio conforme o tipo.
p_ip_conexao	IN	Endereço IP de origem.
p_dispositivo	IN	SMARTTV, SMARTPHONE ou WEB.
p_reproducao_id	OUT	ID do log criado.

```
CREATE PROCEDURE registrar_reproducao(
    IN p_perfil_id BIGINT,
    IN p_tipo_midia VARCHAR(10),
    IN p_midia_id BIGINT,
    IN p_ip_conexao VARCHAR(45),
    IN p_dispositivo VARCHAR(20),
    OUT p_reproducao_id BIGINT
)
BEGIN
    DECLARE v_existe INT DEFAULT 0;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;

    SELECT COUNT(*)
    INTO v_existe
    FROM perfis
    WHERE id = p_perfil_id;

    IF v_existe = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Perfil nao encontrado.';
    END IF;

    IF UPPER(p_tipo_midia) NOT IN ('FILME', 'EPISODIO') THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Tipo de midia invalido. Use FILME ou EPISODIO.';
    END IF;

    IF UPPER(p_dispositivo) NOT IN ('SMARTTV', 'SMARTPHONE', 'WEB') THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Dispositivo invalido.';
    END IF;
```

```

IF UPPER(p_tipo_midia) = 'FILME' THEN

    SELECT COUNT(*)
    INTO v_existe
    FROM filmes
    WHERE id = p_midia_id;

    IF v_existe = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Filme nao encontrado.';
    END IF;

    INSERT INTO logs_reproducao (
        perfil_id,
        tipo_midia,
        filme_id,
        episodio_id,
        ip_conexao,
        dispositivo,
        tempo_assistido_minutos
    )
    VALUES (
        p_perfil_id,
        'FILME',
        p_midia_id,
        NULL,
        p_ip_conexao,
        UPPER(p_dispositivo),
        0
    );

ELSE

    SELECT COUNT(*)
    INTO v_existe
    FROM episodios
    WHERE id = p_midia_id;

    IF v_existe = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Episodio nao encontrado.';
    END IF;

    INSERT INTO logs_reproducao (
        perfil_id,
        tipo_midia,
        filme_id,
        episodio_id,
        ip_conexao,
        dispositivo,
        tempo_assistido_minutos
    )
    VALUES (
        p_perfil_id,
        'EPISODIO',
        NULL,
        p_midia_id,
        p_ip_conexao,
        UPPER(p_dispositivo),
        0
    );
END IF;

SET p_reproducao_id = LAST_INSERT_ID();

COMMIT;
END$$

```

Código 4 - Procedure registrar_reproducao.

Imutabilidade posterior

A Procedure é responsável pela criação do log. Depois de inserido, triggers RN03 impedem UPDATE e DELETE, preservando a trilha de consumo.

5.3 Procedure gerar_faturamento_mensal

A Procedure gerar_faturamento_mensal percorre todas as produtoras por cursor, calcula o consumo mensal de cada uma por meio da Function minutos_assistidos_por_produtores e grava ou atualiza o consolidado em faturamento_produtores.

```
CREATE PROCEDURE gerar_faturamento_mensal(
    IN p_competencia DATE
)
BEGIN
    DECLARE v_fim INT DEFAULT 0;
    DECLARE v_produtores_id BIGINT;
    DECLARE v_minutos BIGINT DEFAULT 0;
    DECLARE v_competencia_normalizada DATE;

    DECLARE cur_produtores CURSOR FOR
        SELECT id
        FROM produtores
        ORDER BY id;

    DECLARE CONTINUE HANDLER FOR NOT FOUND
        SET v_fim = 1;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    SET v_competencia_normalizada =
        STR_TO_DATE(
            DATE_FORMAT(p_competencia, '%Y-%m-01'),
            '%Y-%m-%d'
        );

    START TRANSACTION;

    OPEN cur_produtores;

    leitura_produtores: LOOP

        FETCH cur_produtores
        INTO v_produtores_id;

        IF v_fim = 1 THEN
            LEAVE leitura_produtores;
        END IF;

        SET v_minutos =
            minutos_assistidos_por_produtores(
                v_produtores_id,
                v_competencia_normalizada
            );

        INSERT INTO faturamento_produtores (
            produtores_id,
            competencia,
            minutos_consumidos
        )
        VALUES (
            v_produtores_id,
            v_competencia_normalizada,
            v_minutos
        )
        ON DUPLICATE KEY UPDATE
            minutos_consumidos = VALUES(minutos_consumidos),
            data_ultima_alteracao = CURRENT_TIMESTAMP;

    END LOOP;

    CLOSE cur_produtores;

    COMMIT;
END$$
```

Código 5 - Procedure gerar_faturamento_mensal.

Uso do cursor e handlers

O cursor `cur_produtoras` seleciona os IDs das produtoras em ordem. O `CONTINUE HANDLER FOR NOT FOUND` altera `v_fim` para encerrar o loop quando o `FETCH` não encontrar mais linhas. Paralelamente, o `EXIT HANDLER FOR SQLEXCEPTION` garante rollback de qualquer falha inesperada.

A cláusula `ON DUPLICATE KEY UPDATE` torna o processo idempotente para a combinação `produtora_id + competencia`: uma nova execução da mesma competência atualiza o registro existente em vez de gerar duplicidade.

6. Triggers e regras automatizadas

Trigger	Momento	Tabela	Responsabilidade
tg_limite_perfis	BEFORE INSERT	perfis	Bloqueia o sexto perfil de um assinante.
tg_assinantes_saldo_insert	BEFORE INSERT	assinantes	Impede saldo negativo em novos registros.
tg_assinantes_saldo_update	BEFORE UPDATE	assinantes	Impede saldo negativo em alterações.
tg_assinantes_normaliza_nome	BEFORE INSERT	assinantes	Aplica TRIM + UPPER ao nome.
tg_assinantes_timestamp_update	BEFORE UPDATE	assinantes	Atualiza data_ultima_alteracao.
tg_logs_bloqueia_update	BEFORE UPDATE	logs_reproducao	Impede alteração de eventos de reprodução.
tg_logs_bloqueia_delete	BEFORE DELETE	logs_reproducao	Impede exclusão de eventos de reprodução.
tg_perfis_auditoria_update	AFTER UPDATE	perfis	Registra OLD/NEW e CURRENT_USER().

6.1 Limite de cinco perfis

É executada antes do INSERT porque a regra deve rejeitar a linha antes que ela seja persistida. NEW.assinante_id identifica para qual assinante o novo perfil seria criado. O COUNT(*) verifica quantos perfis já existem e SIGNAL interrompe a operação quando o total é cinco ou mais.

```
CREATE TRIGGER tg_limite_perfis
BEFORE INSERT ON perfis
FOR EACH ROW
BEGIN
    DECLARE v_quantidade INT;

    SELECT COUNT(*)
    INTO v_quantidade
    FROM perfis
    WHERE assinante_id = NEW.assinante_id;

    IF v_quantidade >= 5 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Um assinante pode possuir no maximo 5 perfis.';
    END IF;
END$$
```

Trecho - tg_limite_perfis.

6.2 RN01 - saldo não negativo

As duas triggers protegem o mesmo invariante em momentos diferentes. O uso de BEFORE permite validar NEW.saldo antes da gravação. A proteção é mantida tanto no INSERT quanto no UPDATE para que acessos diretos à tabela não contornem a regra.

```
CREATE TRIGGER tg_assinantes_saldo_insert
BEFORE INSERT ON assinantes
FOR EACH ROW
BEGIN
    IF NEW.saldo < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RN01 violada: o saldo do assinante nao pode ser negativo.';
    END IF;
END$$
```

Trecho - tg_assinantes_saldo_insert.

```
CREATE TRIGGER tg_assinantes_saldo_update
BEFORE UPDATE ON assinantes
FOR EACH ROW
BEGIN
    IF NEW.saldo < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RN01 violada: o saldo do assinante nao pode ser negativo.';
    END IF;
END$$
```

```
END IF;
END$$
```

Trecho - tg_assinantes_saldo_update.

6.3 Normalização de nome

A trigger altera NEW.nome antes do INSERT, removendo espaços externos com TRIM e padronizando o conteúdo com UPPER. Como a transformação ocorre antes da persistência, todos os novos assinantes passam a obedecer ao mesmo padrão.

```
CREATE TRIGGER tg_assinantes_normaliza_nome
BEFORE INSERT ON assinantes
FOR EACH ROW
BEGIN
    SET NEW.nome = UPPER(TRIM(NEW.nome));
END$$
```

Trecho - tg_assinantes_normaliza_nome.

6.4 Atualização automática de timestamp

É uma trigger BEFORE UPDATE porque precisa atribuir CURRENT_TIMESTAMP à própria linha que será gravada. Isso centraliza a manutenção do campo data_ultima_alteracao.

```
CREATE TRIGGER tg_assinantes_timestamp_update
BEFORE UPDATE ON assinantes
FOR EACH ROW
BEGIN
    SET NEW.data_ultima_alteracao = CURRENT_TIMESTAMP;
END$$
```

Trecho - tg_assinantes_timestamp_update.

6.5 RN03 - logs imutáveis

As triggers são executadas BEFORE UPDATE e BEFORE DELETE e sempre emitem SIGNAL. O comportamento protege os logs como evidência histórica de consumo, impedindo alteração ou remoção após a criação.

```
CREATE TRIGGER tg_logs_bloqueia_update
BEFORE UPDATE ON logs_reproducao
FOR EACH ROW
BEGIN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'RN03: reproducoes sao imutaveis. UPDATE proibido.';
END$$
```

Trecho - tg_logs_bloqueia_update.

```
CREATE TRIGGER tg_logs_bloqueia_delete
BEFORE DELETE ON logs_reproducao
FOR EACH ROW
BEGIN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'RN03: reproducoes sao imutaveis. DELETE proibido.';
END$$
```

Trecho - tg_logs_bloqueia_delete.

6.6 Auditoria de perfis

A auditoria é AFTER UPDATE porque o registro de auditoria deve ser criado somente depois que a alteração principal for considerada válida. OLD representa o estado anterior; NEW representa o estado final. JSON_OBJECT mantém ambos em formato estruturado, e CURRENT_USER() identifica a conta efetivamente utilizada pelo SGBD.

```
CREATE TRIGGER tg_perfis_auditoria_update
AFTER UPDATE ON perfis
FOR EACH ROW
BEGIN
    INSERT INTO auditoria_log (
        tabela,
        operacao,
        usuario,
        valor_antigo,
        valor_novo
    )
```



```
VALUES (
    'perfis',
    'UPDATE',
    CURRENT_USER(),

    JSON_OBJECT(
        'id', OLD.id,
        'assinante_id', OLD.assinante_id,
        'nome_exibicao', OLD.nome_exibicao
    ),

    JSON_OBJECT(
        'id', NEW.id,
        'assinante_id', NEW.assinante_id,
        'nome_exibicao', NEW.nome_exibicao
    )
);
END$$
```

Trecho - tg_perfis_auditoria_update.

6.7 Trigger x CHECK constraint

A regra de saldo demonstra dois mecanismos complementares. O CHECK chk_saldo_positivo é declarativo e expressa diretamente o invariante saldo ≥ 0 . As triggers, por sua vez, permitem mensagem de erro específica e podem evoluir para regras condicionais mais complexas.

Critério	CHECK	TRIGGER
Natureza	Declarativa	Procedural
Momento	Validado pelo SGBD na gravação	Executado em evento INSERT/UPDATE/DELETE
Mensagem personalizada	Limitada	Sim, por SIGNAL
Complexidade adequada	Regras simples de domínio	Regras condicionais e efeitos adicionais
Uso no projeto	saldo ≥ 0 ; durações e minutos	RN01, RN03, auditoria, timestamp, normalização, limite de perfis

Defesa em profundidade

Manter CHECK e trigger para saldo é redundante do ponto de vista da condição matemática, mas aumenta a proteção e fornece uma mensagem de negócio explícita. Em uma revisão arquitetural, a redundância deve ser documentada para que não seja interpretada como duplicação acidental.

7. Tratamento de exceções e transações

As Procedures transacionais seguem o padrão START TRANSACTION → validações → alterações → COMMIT. Um EXIT HANDLER FOR SQLEXCEPTION executa ROLLBACK e RESIGNAL, preservando a atomicidade e devolvendo o erro original ou a mensagem criada por SIGNAL.

Situação	Objeto	Mecanismo	Efeito
Valor de mensalidade inválido	realizar_cobranca_mensal	SIGNAL 45000	Interrompe e desfaz a transação.
Assinante inexistente	realizar_cobranca_mensal	SIGNAL 45000	Nenhum débito é realizado.
Saldo insuficiente	realizar_cobranca_mensal	SIGNAL 45000	Saldo permanece inalterado.
Perfil, filme ou episódio inexistente	registrar_reproducao	SIGNAL 45000	Nenhum log é criado.
Tipo de mídia/dispositivo inválido	registrar_reproducao	SIGNAL 45000	Entrada rejeitada.
Erro SQL inesperado	Procedures transacionais	EXIT HANDLER + ROLLBACK + RESIGNAL	Restaura consistência e propaga o erro.
Fim do cursor	gerar_faturamento_mensal	CONTINUE HANDLER FOR NOT FOUND	Encerra o loop normalmente.

7.1 Concorrência na cobrança

O SELECT ... FOR UPDATE bloqueia a linha do assinante durante a transação. Isso evita que duas cobranças concorrentes leiam simultaneamente o mesmo saldo e debitem com base em um valor desatualizado. O bloqueio é liberado ao final do COMMIT ou ROLLBACK.

8. Índices e desempenho

Os índices adicionados foram escolhidos a partir dos principais padrões de consulta do sistema: recuperação de itens pendentes por perfil, análise temporal dos logs, agrupamento por dispositivo, busca de logs por perfil, segmentação por UF e acesso de conteúdos por produtora.

Índice	Tabela	Colunas	Uso esperado
idx_historico_perfil_pendente	historicos_reproducao	perfil_id, concluido, data_hora_atualizacao	Continuar assistindo.
idx_logs_data_acesso	logs_reproducao	data_hora_acesso	Filtros mensais e análise temporal.
idx_logs_dispositivo	logs_reproducao	dispositivo	Métricas por dispositivo.
idx_logs_perfil	logs_reproducao	perfil_id	Consumo por perfil.
idxassinantes_uf	assinantes	uf	Segmentação regional.
idx_conteudos_produtores	conteudos	produtora_id	Consumo/faturamento por produtora.

Observação

Índices melhoram leituras, mas adicionam custo de manutenção em INSERT/UPDATE. O conjunto utilizado é pequeno e direcionado às consultas de negócio mais frequentes.

9. View LGPD

A view `vw_analise_engajamento` disponibiliza informações agregadas de uso sem expor diretamente os principais dados pessoais. O nome é substituído por `CONFIDENCIAL`, o CPF é totalmente mascarado e o e-mail mantém apenas dois caracteres do usuário e o domínio.

```
CREATE VIEW vw_analise_engajamento AS
SELECT
    p.id AS perfil_id,
    'CONFIDENCIAL' AS nome,

    CONCAT(
        LEFT(a.email, 2),
        '***@',
        SUBSTRING_INDEX(a.email, '@', -1)
    ) AS email_mascarado,

    '***.***.***-**' AS cpf,

    calcular_idade(a.data_nascimento) AS idade,

    a.uf,

    COUNT(l.id) AS total_reproducoes,

    COALESCE(
        SUM(l.tempo_assistido_minutos),
        0
    ) AS minutos_consumidos

FROM assinantes a

INNER JOIN perfis p
    ON p.assinante_id = a.id

LEFT JOIN logs_reproducao l
    ON l.perfil_id = p.id

GROUP BY
    p.id,
    a.data_nascimento,
    a.uf,
    a.email;
```

Código 6 - View de análise de engajamento com redução de exposição de dados.

Dado original	Tratamento na view	Objetivo
Nome	CONFIDENCIAL	Evitar identificação direta.
E-mail	Dois caracteres + *** + domínio	Reduzir exposição mantendo referência operacional.
CPF	***.***.***-**	Ocultar integralmente o identificador.
Data de nascimento	Convertida em idade	Minimizar dado pessoal mantendo utilidade analítica.
Reproduções/minutos	Agregados	Permitir análise de engajamento.

10. Segurança e privilégios

A solução define dois usuários de banco com responsabilidades distintas. `application_user` representa a aplicação e recebe somente permissões necessárias ao funcionamento; `auditoria_user` recebe acesso de leitura aos registros de auditoria e à view analítica.

Usuário	Privilégios principais	Justificativa
<code>application_user</code>	SELECT/INSERT/UPDATE em tabelas operacionais específicas; SELECT no catálogo e faturamento; EXECUTE em Functions e Procedures.	Permite uso da aplicação sem privilégios administrativos.
<code>auditoria_user</code>	SELECT em <code>logs_reproducao</code> , <code>auditoria_log</code> e <code>vw_analise_engajamento</code> .	Perfil dedicado à consulta e fiscalização.

10.1 Menor privilégio

O ORM foi configurado para utilizar `application_user` em vez de `root`. Essa escolha é superior do ponto de vista de segurança porque limita o impacto de falhas na aplicação. O usuário não recebe DELETE em `logs_reproducao` nem permissões administrativas sobre o SGBD.

Para permitir a conferência do faturamento pela aplicação, foi incluído SELECT em `faturamento_producao`. A escrita permanece encapsulada na Procedure `gerar_faturamento_mensal`; não foi concedido INSERT, UPDATE ou DELETE direto nessa tabela.

10.2 Credenciais de laboratório

Atenção

As senhas presentes no script SQL são adequadas apenas para ambiente acadêmico/local. Em um ambiente real, credenciais devem ser fortes, secretas, rotacionáveis e armazenadas fora do código-fonte por variáveis de ambiente ou cofre de segredos.

11. Dados de teste e consultas analíticas

O conjunto de dados de teste inclui três assinantes, quatro perfis iniciais, preferências, três produtoras, três conteúdos, um filme, duas séries, três episódios, históricos de consumo e quatro logs datados em julho de 2026. A escolha de datas fixas torna os resultados de faturamento reprodutíveis.

Produtora	Conteúdo	Tipo	Consumo de teste em julho/2026
Netflix Studios	Dark	Série	55 + 58 = 113 minutos
Warner Bros	Interestelar	Filme	169 minutos
Disney	Loki	Série	50 minutos

11.1 Consultas implementadas

- Continuar assistindo: recupera históricos não concluídos por perfil.
- Consumo por produtora: agrega minutos e horas no mês.
- Tráfego por região: cruza UF, dispositivo, acessos e minutos.
- Engajamento por idade: cria faixas etárias reutilizando calcular_idade.

12. Plano de validação e resultados

Os testes foram executados em duas frentes: diretamente no SGBD, para validar cada objeto isoladamente, e pela camada TypeScript/Drizzle, para comprovar integração real da aplicação com o banco.

Teste	Esperado	Obtido	Status
Function calcular_idade	22	22	PASS
Function minutos por produtora	113	113	PASS
Cobrança válida SQL	Saldo 100 → 70	Saldo 70	PASS
Cobrança sem saldo	Erro + rollback	Erro e saldo preservado	PASS
Registrar reprodução SQL	Novo log	ID retornado e log criado	PASS
Perfil inexistente	Erro + rollback	Perfil nao encontrado	PASS
Faturamento mensal	113 / 169 / 50	113 / 169 / 50	PASS
RN01 INSERT negativo	Bloqueio	Bloqueado	PASS
RN01 UPDATE negativo	Bloqueio	Bloqueado	PASS
RN03 UPDATE de log	Bloqueio	Bloqueado	PASS
RN03 DELETE de log	Bloqueio	Bloqueado	PASS
Auditoria de perfil	OLD/NEW gravados	Registro gerado	PASS
Timestamp	Atualização automática	Campo atualizado	PASS
Normalização de nome	TESTE DA SILVA	TESTE DA SILVA	PASS
Limite de perfis	Sexto perfil rejeitado	Rejeitado	PASS

Critério de aceite

Um teste é considerado aprovado quando o comportamento observado coincide com a regra definida e, em cenários de erro, não há alteração parcial de dados.

13. Mapeamento ORM com Drizzle

13.1 Escolha do Drizzle ORM

O diferencial ORM foi implementado com Drizzle por ser uma solução TypeScript fortemente tipada e compatível com bancos de dialeto MySQL. A escolha também aproxima a entrega do conteúdo estudado em sala, facilitando a defesa técnica do projeto.

13.2 Estratégia database-first

O banco já existia e continha regras relevantes; portanto, o ORM não foi usado para criar ou alterar o schema. O comando `drizzle-kit pull` introspectou a estrutura existente e gerou `schema.ts`, `relations.ts`, `migration de baseline` e `snapshot`. Isso preserva o SQL original como fonte de verdade do modelo.

Item introspectado	Quantidade
Tabelas	12
Colunas	67
Índices	19
Foreign Keys	13
CHECK constraints	12
Views	1

13.3 Mapeamento das entidades

Banco	Objeto Drizzle	Relacionamento principal
assinantes	assinantes	1:N perfis
perfis	perfis	N:1 assinantes; 1:N históricos e reproduções
preferencias_perfis	preferenciasPerfis	N:1 perfil
produtoras	produtoras	1:N conteúdos e faturamentos
conteudos	conteudos	N:1 produtora; 1:0..1 filme/série
filmes	filmes	1:1 conteúdo
series	series	1:1 conteúdo; 1:N episódios
episodios	episodios	N:1 série
historicos_reproducao	historicosReproducao	N:1 perfil; filme/episódio opcional
logs_reproducao	logsReproducao	N:1 perfil; associação com mídia e atributos
faturamento_produtoras	faturamentoProdutoras	N:1 produtora
auditoria_log	auditoriaLog	Registro independente de auditoria

13.4 Conexão com o banco

```
import "dotenv/config";

import { createPool } from "mysql2";
import { drizzle } from "drizzle-orm/mysql2";

import { relations } from "../relations";

const databaseUrl = process.env.APP_DATABASE_URL;

if (!databaseUrl) {
```



```

    throw new Error("APP_DATABASE_URL nao definida.");
  }

  export const pool = createPool(databaseUrl);

  export const db = drizzle({
    client: pool,
    relations,
  });

```

Código 7 - Inicialização da conexão Drizzle com pool mysql2.

A conexão utiliza APP_DATABASE_URL para que o código da aplicação se autentique como application_user. A variável administrativa DATABASE_URL pode permanecer reservada ao Drizzle Kit durante introspecção e atividades de desenvolvimento.

13.5 Consultas ORM

No ambiente MariaDB, foi utilizado o Query Builder do Drizzle com joins explícitos. Essa abordagem continua sendo ORM/query builder tipado, mas evita a Relational Query API que, na versão testada, gera LEFT JOIN LATERAL incompatível com o servidor MariaDB utilizado.

```

const assinantesPerfis = await db
  .select({
    assinanteId: assinantes.id,
    nome: assinantes.nome,
    email: assinantes.email,
    saldo: assinantes.saldo,
    perfilId: perfis.id,
    nomeExibicao: perfis.nomeExibicao,
  })
  .from(assinantes)
  .leftJoin(perfis, eq(assinantes.id, perfis.assinanteId));

```

Código 8 - Exemplo de consulta ORM com relacionamento Assinante 1:N Perfil.

13.6 Associação com atributos e especialização

logs_reproducao é mapeada como uma associação de reprodução com atributos próprios: IP, dispositivo, data/hora e minutos assistidos. Além disso, a camada ORM reproduz a especialização Conteudo → Filme/Serie e Serie → Episodio, permitindo navegar pelo domínio sem eliminar a estrutura relacional definida no banco.

14. Integração das rotinas SQL pela aplicação

Stored Procedures e Functions não são reimplementadas no ORM. A aplicação utiliza uma conexão mysql2 dedicada quando precisa executar CALL, obter parâmetros OUT ou garantir que variáveis de sessão como @novo_saldo permaneçam na mesma sessão. Após a execução, o Drizzle pode consultar os registros resultantes.

14.1 Functions

```
const [idade] = await connection.query<RowDataPacket[]>(
  "SELECT calcular_idade(?) AS idade",
  ["2004-05-20"]
);

const [minutos] = await connection.query<RowDataPacket[]>(
  "SELECT minutos_assistidos_por_produtores(?, ?) AS minutos",
  [1, "2026-07-01"]
);
```

Código 9 - Invocação das Functions pela camada de dados.

14.2 Cobrança mensal pela aplicação

```
await connection.query("SET @novo_saldo = NULL");
await connection.query(
  "CALL realizar_cobranca_mensal(?, ?, @novo_saldo)",
  [4, 5.00]
);
const [resultado] = await connection.query<RowDataPacket[]>(
  "SELECT @novo_saldo AS novo_saldo"
);
```

Código 10 - Chamada da Procedure de cobrança pela aplicação.

No teste integrado, o assinante TESTE DA SILVA passou de saldo 50,00 para 45,00. O parâmetro OUT retornou 45,00 e uma consulta posterior com Drizzle confirmou o mesmo valor.

14.3 Registro de reprodução pela aplicação

A aplicação criou uma reprodução do tipo FILME para o perfil 1, usando o filme 1, IP 200.100.50.20 e dispositivo WEB. A Procedure retornou id_reproducao = 6; em seguida, o Drizzle consultou esse mesmo ID e confirmou os atributos gravados.

14.4 Faturamento mensal pela aplicação

A aplicação invocou gerar_faturamento_mensal para julho de 2026 e depois consultou faturamento_produtores por Drizzle. O resultado confirmou 113 minutos para Netflix Studios, 169 para Warner Bros e 50 para Disney.

Encapsulamento preservado

A aplicação não sabe como o cursor funciona, como os minutos são calculados nem como a transação é controlada. Esses detalhes permanecem no banco. Esse é exatamente o benefício de manter regras críticas encapsuladas na camada de dados.

15. Compatibilidade MariaDB/MySQL e decisões técnicas

15.1 SGBD-alvo x ambiente real

O cabeçalho do script identifica MySQL 8.0 como SGBD-alvo. Entretanto, mensagens retornadas pelo servidor durante os testes ORM identificaram o ambiente como MariaDB. Os objetos SQL empregados - procedures, functions, cursores, SIGNAL, handlers, triggers, views e constraints - funcionaram no ambiente utilizado.

Recomendação para a entrega

Antes da submissão, alinhar a identificação do SGBD com a orientação do professor. Uma forma tecnicamente precisa é registrar “SGBD-alvo: MySQL 8.0; ambiente local validado: MariaDB compatível com o dialeto utilizado”.

15.2 Query Builder em vez da Relational Query API

A API relacional mais recente do Drizzle gerou SQL com LEFT JOIN LATERAL. O servidor MariaDB utilizado rejeitou essa sintaxe no formato gerado. A solução adotada foi o Query Builder tipado do Drizzle com leftJoin/innerJoin explícitos, que é compatível com o ambiente e continua demonstrando mapeamento objeto-relacional e construção de consultas pela biblioteca.

15.3 Pool mysql2

A conexão foi criada com createPool do pacote mysql2 e passada ao Drizzle. Para rotinas com parâmetros OUT e variáveis de sessão, uma conexão é adquirida do mesmo pool e utilizada do SET ao SELECT final, garantindo que toda a sequência permaneça na mesma sessão do banco.

15.4 Bônus PostgreSQL

O requisito opcional de trigger FOR EACH STATEMENT é específico do PostgreSQL e não foi implementado. A solução usa triggers FOR EACH ROW, compatíveis com o SGBD escolhido. Essa diferença deve ser declarada explicitamente, e não tratada como requisito ausente do escopo obrigatório.

16. Matriz de rastreabilidade do enunciado

Requisito	Implementação	Status
Cobrança mensal com IN/OUT, SELECT INTO e erro de saldo	realizar_cobranca_mensal	Atendido
Registro de reprodução com transação e rollback	registrar_reproducao	Atendido
Faturamento mensal com cursor e NOT FOUND handler	gerar_faturamento_mensal	Atendido
Function minutos por produtora	minutos_assistidos_por_produtora	Atendido
Function calcular idade e uso em view	calcular_idade + vw_analise_engajamento	Atendido
RN01 saldo negativo INSERT/UPDATE	triggers + CHECK	Atendido
RN03 reprodução imutável	triggers UPDATE/DELETE	Atendido
Auditoria com usuário e OLD/NEW	tg_perfis_auditoria_update	Atendido
Timestamp automático	tg_assinantes_timestamp_update	Atendido
Nome normalizado	tg_assinantes_normaliza_nome	Atendido
Tratamento de exceções	SIGNAL, EXIT/CONTINUE handlers, ROLLBACK	Atendido
ORM - entidades e relações	Drizzle schema.ts + relations.ts	Atendido
ORM - associação com atributos	logs_reproducao	Atendido
ORM - especialização filme/série/episódio	conteudos, filmes, series, episodios	Atendido
ORM - chamada de Functions/Procedures	procedimentos.ts	Atendido
Bônus FOR EACH STATEMENT PostgreSQL	Não aplicável ao SGBD escolhido	Fora do escopo

17. Guia de execução

17.1 Banco de dados

10. Abrir o cliente SQL com uma conta administrativa.
11. Executar o script streamflow.sql completo para recriar o banco e todos os objetos.
12. Executar os testes manuais individualmente quando a intenção for observar mensagens de erro.
13. Conferir usuários e privilégios com SHOW GRANTS.

17.2 ORM

```
cd orm
npm install
npm run check
npm run consultas
npm run procedimentos -- funcoes
npm run procedimentos -- cobranca
npm run procedimentos -- reproducao
npm run procedimentos -- faturamento
```

Comandos principais da camada ORM.

17.3 Introspecção do banco

```
npx drizzle-kit pull
```

Comando database-first utilizado para gerar schema e relações a partir do banco existente.

Não executar na entrega

Evitar drizzle-kit push ou migrate sobre o banco já construído pelo script SQL, salvo se houver intenção explícita de migrar a responsabilidade do schema para o Drizzle. Neste projeto, o SQL permanece a fonte de verdade.

18. Conclusão

A evolução do StreamFlow demonstra uma arquitetura de banco de dados que vai além da criação de tabelas. As regras mais sensíveis foram centralizadas no SGBD, transações protegem operações compostas, exceções são tratadas de forma previsível, logs de reprodução são imutáveis, alterações de perfil são auditadas e a segurança segue uma divisão de privilégios coerente com as responsabilidades de cada usuário.

A integração com Drizzle ORM comprova que a aplicação pode consumir o modelo relacional sem duplicar as regras críticas. O ORM mapeia entidades e relacionamentos, fornece consultas tipadas e atua em conjunto com mysql2 para invocar Stored Procedures e Functions. O resultado é uma solução em que integridade, encapsulamento e rastreabilidade permanecem no banco, enquanto a aplicação ganha produtividade e segurança de tipos.

Os testes executados demonstram consistência entre a validação direta no SGBD e a integração pela aplicação: as mesmas Functions retornam os mesmos valores, as Procedures produzem os resultados esperados e o faturamento mensal permanece determinístico para o conjunto de dados de julho de 2026.

Estado final

Implementação pronta para entrega técnica.

Apêndice A - Estrutura de entrega

A entrega recomendada separa o script SQL da camada ORM e não inclui dependências instaladas ou credenciais reais.

```
StreamFlow/
├── streamflow.sql
├── Documentacao_Tecnica_StreamFlow.pdf
├── orm/
│   ├── drizzle/
│   ├── src/
│   │   ├── db/
│   │   │   ├── index.ts
│   │   │   ├── relations.ts
│   │   │   └── schema.ts
│   │   ├── consultas.ts
│   │   └── procedimentos.ts
│   ├── .env.example
│   ├── .gitignore
│   ├── drizzle.config.ts
│   ├── package.json
│   ├── package-lock.json
│   └── tsconfig.json
```

Estrutura final recomendada.

Item	Enviar?	Observação
streamflow.sql	Sim	Fonte de verdade do banco.
Documentação PDF	Sim	Relatório técnico final.
orm/src	Sim	Código TypeScript.
orm/drizzle	Sim	Baseline/snapshot gerados pela introspecção.
package.json / package-lock.json	Sim	Permite restaurar dependências.
.env.example	Sim	Somente placeholders.
.env	Não	Contém credenciais.
node_modules	Não	Recriado com npm install.